# RV32I Base Integer ISA — Instruction Reference

Bit-field layout, opcodes, funct3/funct7, and encoding rules for every RV32I instruction type.

## 1. Instruction Formats (32-bit, fields in bit positions)

| Type | 31:25 | 24:20 | 19:15 | 14:12 | 11:7 | 6:0 |
|---|---|---|---|---|---|---|
| R | funct7 | rs2 | rs1 | funct3 | rd | opcode |
| I | imm[11:0] | | rs1 | funct3 | rd | opcode |
| S | imm[11:5] | rs2 | rs1 | funct3 | imm[4:0] | opcode |
| B | imm[12\|10:5] | rs2 | rs1 | funct3 | imm[4:1\|11] | opcode |
| U | imm[31:12] | | | | rd | opcode |
| J | imm[20\|10:1\|11\|19:12] | | | | rd | opcode |

Note on immediate bit ordering: B-type and J-type immediates are NOT stored contiguously — you must manually reassemble bits in the correct order and sign-extend from the MSB. This is the #1 source of bugs in immediate generators. Verify against the spec bit-by-bit.

## 2. Base Opcodes (bits [6:0], bottom 2 bits always 11 for RV32I)

| Opcode (bin) | Opcode (hex) | Format | Used by |
|---|---|---|---|
| 0110111 | 0x37 | U | LUI |
| 0010111 | 0x17 | U | AUIPC |
| 1101111 | 0x6F | J | JAL |
| 1100111 | 0x67 | I | JALR |
| 1100011 | 0x63 | B | BEQ, BNE, BLT, BGE, BLTU, BGEU |
| 0000011 | 0x03 | I | LB, LH, LW, LBU, LHU |
| 0100011 | 0x23 | S | SB, SH, SW |
| 0010011 | 0x13 | I | ADDI, SLTI, SLTIU, XORI, ORI, ANDI, SLLI, SRLI, SRAI |
| 0110011 | 0x33 | R | ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR, AND |
| 0001111 | 0x0F | I (fence) | FENCE |
| 1110011 | 0x73 | I (system) | ECALL, EBREAK |

## 3. Full RV32I Instruction Table

### R-type (opcode 0110011)

| Instr | funct3 | funct7 | Operation |
|---|---|---|---|
| ADD | 000 | 0000000 | rd = rs1 + rs2 |
| SUB | 000 | 0100000 | rd = rs1 - rs2 |
| SLL | 001 | 0000000 | rd = rs1 << rs2[4:0] |
| SLT | 010 | 0000000 | rd = (rs1 <s rs2) ? 1 : 0  (signed) |
| SLTU | 011 | 0000000 | rd = (rs1 <u rs2) ? 1 : 0  (unsigned) |
| XOR | 100 | 0000000 | rd = rs1 ^ rs2 |
| SRL | 101 | 0000000 | rd = rs1 >> rs2[4:0]  (logical) |
| SRA | 101 | 0100000 | rd = rs1 >> rs2[4:0]  (arithmetic) |
| OR | 110 | 0000000 | rd = rs1 \| rs2 |
| AND | 111 | 0000000 | rd = rs1 & rs2 |

### I-type — ALU immediate (opcode 0010011)

| Instr | funct3 | funct7 (shifts only) | Operation |
|---|---|---|---|
| ADDI | 000 | - | rd = rs1 + sext(imm) |
| SLTI | 010 | - | rd = (rs1 <s sext(imm)) ? 1 : 0 |
| SLTIU | 011 | - | rd = (rs1 <u sext(imm)) ? 1 : 0 |
| XORI | 100 | - | rd = rs1 ^ sext(imm) |
| ORI | 110 | - | rd = rs1 \| sext(imm) |
| ANDI | 111 | - | rd = rs1 & sext(imm) |
| SLLI | 001 | 0000000 (imm[11:5]) | rd = rs1 << shamt(imm[4:0]) |
| SRLI | 101 | 0000000 (imm[11:5]) | rd = rs1 >> shamt  (logical) |
| SRAI | 101 | 0100000 (imm[11:5]) | rd = rs1 >> shamt  (arithmetic) |

### I-type — Loads (opcode 0000011)

| Instr | funct3 | Operation |
|---|---|---|
| LB | 000 | rd = sext(mem[rs1+imm][7:0]) |
| LH | 001 | rd = sext(mem[rs1+imm][15:0]) |
| LW | 010 | rd = mem[rs1+imm][31:0] |
| LBU | 100 | rd = zext(mem[rs1+imm][7:0]) |
| LHU | 101 | rd = zext(mem[rs1+imm][15:0]) |

### I-type — JALR (opcode 1100111) / System (opcode 1110011)

| Instr | funct3 | imm (system) | Operation |
|---|---|---|---|
| JALR | 000 | - | rd = pc+4; pc = (rs1+sext(imm)) & ~1 |
| ECALL | 000 | 0x000 | Environment call (trap) |
| EBREAK | 000 | 0x001 | Breakpoint (trap) |

### S-type — Stores (opcode 0100011)

| Instr | funct3 | Operation |
|---|---|---|
| SB | 000 | mem[rs1+imm][7:0] = rs2[7:0] |
| SH | 001 | mem[rs1+imm][15:0] = rs2[15:0] |
| SW | 010 | mem[rs1+imm][31:0] = rs2[31:0] |

### B-type — Branches (opcode 1100011)

| Instr | funct3 | Operation (branch taken if true, target = pc + imm) |
|---|---|---|
| BEQ | 000 | rs1 == rs2 |
| BNE | 001 | rs1 != rs2 |
| BLT | 100 | rs1 <s rs2  (signed) |
| BGE | 101 | rs1 >=s rs2  (signed) |
| BLTU | 110 | rs1 <u rs2  (unsigned) |
| BGEU | 111 | rs1 >=u rs2  (unsigned) |

### U-type (LUI opcode 0110111, AUIPC opcode 0010111)

| Instr | Operation |
|---|---|
| LUI | rd = imm[31:12] << 12   (loads upper 20 bits, lower 12 = 0) |
| AUIPC | rd = pc + (imm[31:12] << 12)   (pc-relative addressing) |

### J-type (JAL opcode 1101111)

| Instr | Operation |
|---|---|
| JAL | rd = pc + 4;  pc = pc + imm  (imm is sign-extended, 20-bit range, word*2 aligned) |

## 4. Register ABI names (x0-x31)

| Reg | ABI | Use | Reg | ABI | Use |
|---|---|---|---|---|---|
| x0 | zero | hardwired 0 | x16-17 | a6-a7 | arg/return |
| x1 | ra | return addr | x18-27 | s2-s11 | saved regs |
| x2 | sp | stack pointer | x28-31 | t3-t6 | temporaries |
| x3 | gp | global pointer | | | |
| x4 | tp | thread pointer | | | |
| x5-7 | t0-t2 | temporaries | | | |
| x8 | s0/fp | saved/frame ptr | | | |
| x9 | s1 | saved reg | | | |
| x10-15 | a0-a5 | args/return | | | |

## 5. Design checklist before you code the datapath

1. Decode opcode first -> branches into format type -> then use funct3 (and funct7 for R-type/shift-immediates) to pick exact operation.
2. sext() means sign-extend using the immediate's MSB, not zero-fill. Getting this wrong silently breaks negative immediates.
3. B-type and J-type targets are PC-relative and the immediate LSB is implicitly 0 (never encoded) — instructions are 2-byte aligned.
4. rd = x0 writes must be discarded (x0 is always 0) — decide where in your datapath you enforce this.
5. funct7 only matters for R-type and shift-immediates (SLLI/SRLI/SRAI) — don't build a decoder that checks funct7 for every I-type, that's wasted logic and a common design mistake.